STUDY TEXTBOOK

BOOK TITLE

Subtitle

First edition · 10 October 2026

Contents

1 Style smoke test **1**

1.1 Environments . . . 1

1.2 A graph . . . 2

1.3 A diagram, step by step . . . 2

1.4 A circuit . . . 2

1.5 Code with output . . . 3

Notes . . . 3

2 How a program becomes a running thing **4**

2.1 The four-stage journey . . . 4

2.2 Object files and ELF sections . . . 6

2.3 Symbols and relocation . . . 7

2.4 The loader and `execve` . . . 8

2.5 Where Book 5's CPU meets real silicon . . . 9

2.6 A map of the book . . . 10

2.7 Common mistakes . . . 10

Exercises . . . 11

Summary . . . 12

Further reading . . . 13

Watch instead . . . 13

Notes . . . 13

3 How the machine sees data **14**

3.1 Bytes, words and addresses . . . 14

3.2 Two's complement, in one page . . . 15

3.3 Endianness: which byte goes first . . . 15

3.4 Alignment and struct layout . . . 16

3.5 IEEE-754 in one breath . . . 17

3.6 Pointers as typed addresses; arrays vs pointers . . . 18
3.7 Common mistakes . . . 19
Exercises . . . 19
Summary . . . 21
Further reading . . . 21
Watch instead . . . 21
Notes . . . 22

4 RV32I assembly I: arithmetic, memory, control 23
4.1 The register file and the ABI names . . . 23
4.2 Six formats, one discipline . . . 24
4.3 Loads, stores and addressing . . . 25
4.4 Branches and comparisons . . . 26
4.5 Assembler directives and running it . . . 26
4.6 Common mistakes . . . 28
Exercises . . . 28
Summary . . . 30
Further reading . . . 30
Watch instead . . . 31
Notes . . . 31

5 Functions and the Stack 32
5.1 The contract in one paragraph . . . 32
5.2 The stack frame . . . 32
5.3 Caller-saved, callee-saved . . . 33
5.4 What a call costs . . . 34
5.5 Reading real compiler output . . . 35
5.6 PIC in one page . . . 36
5.7 Common mistakes . . . 36
5.8 Summary . . . 37
5.9 Exercises . . . 38

CHAPTER 1

Style smoke test

This chapter exists only to prove that every environment, figure type and code block compiles. Delete it when the first real chapter exists. The model for structure is [1].

1.1 Environments

Definition 1.1 (Function)**.** A *function* $f : A \to B$ assigns to each $a \in A$ exactly one $f(a) \in B$.

Theorem 1.2 (Sum of the first n integers)**.** *For every* $n \geq 1$, $\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$.

Proof. By induction on n. For $n = 1$ both sides are 1. If it holds for n, then adding $n+1$ gives $\frac{n(n+1)}{2} + n + 1 = \frac{(n+1)(n+2)}{2}$. ∎

Lemma 1.3. *A small lemma.*

Proposition 1.4. *A small proposition.*

Example 1.5. Worked example: strategy first, then solution.

Solution. The answer is 42.

Remark. A remark.

Exercise 1.1 (easy)**.** Show that $\sum_{k=1}^{n} (2k-1) = n^2$.

Exercise 1.2 (hard)**.** Prove the theorem above without induction.

PRINCIPLE

A principle box.

DECISION RULE

A rule box.

TRIPWIRE

A warning box.

DEFINITION OF DONE

A done box.

Checkpoint 1.1 *Try it.* Run the code in Listing 1.1 and predict the output first.

1.2 A graph

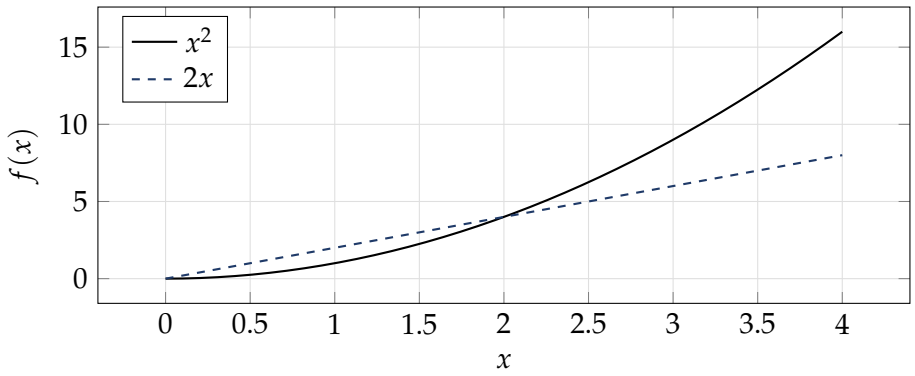


Figure 1.1: A pgfplots graph. Notice that x^2 overtakes $2x$ after $x = 2$. Line styles, not only colour, tell the curves apart.

1.3 A diagram, step by step

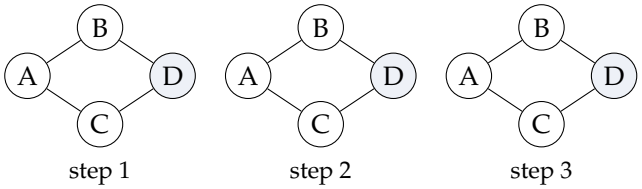


Figure 1.2: Small multiples show the state of an algorithm at three successive steps.

1.4 A circuit

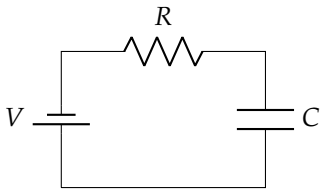


Figure 1.3: An RC circuit drawn with circuitikz. Every circuit in the book is drawn this way.

1.5 Code with output

```
# why: show the loop invariant, not just the loop
total = 0
for k in range(1, 6):
    total += k
print(total)
```

Listing 1.1: A short listing with line numbers.

Output of a real run:

```
15
```

Name	Meaning	Cost
Sort	Order the items	$O(n \log n)$
Scan	Visit each item once	$O(n)$

Table 1.1: Cheat sheet table: each chapter ends its main text with one of these.

Notes

Sources are cited by number [1].

CHAPTER 2

How a program becomes a running thing

You wrote a CPU. In Book 5 you designed an 8-bit ALU, wired it into a datapath, and drove it with a two-state FSM until Fibonacci numbers came out of a serial pin. That machine is small enough to hold in your head — 302 synthesis cells, ten opcodes, two states — but it is not a toy. The CPU you are using right now runs *more* code than yours, not *different* code. Somewhere between `int main(void)` and the transistors of your cpu8 there is a journey every program takes, and this book is about travelling it on foot. This chapter walks the first mile: how source text becomes a file of machine code, how that file gets into memory, and where your cpu8 meets the real silicon. Along the way you will compile, disassemble and link *real* binaries and read their real output — nothing in this chapter is simulated.

2.1 The four-stage journey

Type `cc sum.c -o sum` and four programs run in sequence, each handing a new file to the next. The names differ between toolchains — `gcc`, `clang`, `cc` — the shape does not:

1. **Preprocess** — text in, text out. `#include` splices header files in place; `#define` substitutes macros; conditionals (`#ifdef`) delete code before the compiler ever sees it. Output: a "translation unit", still plain C.
2. **Compile** — the translation unit becomes assembly: mnemonics, register names, labels — the same language your cpu8 assembler spoke, with a bigger vocabulary. Output: `.s`.
3. **Assemble** — mnemonics become machine-code bytes plus bookkeeping (which bytes are code, which are data, which names are exported). Output: an *object file*, `.o` — already ELF, just not yet executable.
4. **Link** — object files are stitched together: symbols resolved, sections merged, addresses assigned, one entry point recorded. Output: an executable ELF.

Example 2.1 (The pipeline, hands on)**.** Here is the whole journey on `code/ch01_sum.c`, a six-line freestanding sum. Each stage was run on the dev box (Apple clang 21, targeting x86-64 Linux so the output is real ELF):

```
clang -E ch01_sum.c > ch01_sum.i # 1: preprocess
clang -S ch01_sum.i # 2: compile -> ch01_sum.s
clang -c ch01_sum.s # 3: assemble -> ch01_sum.o
ld.lld -o ch01_sum.elf ch01_start.o ch01_sum.o # 4: link
```

Stage 2 emits assembly you can already read. Real output from `clang -S` (first lines of `sum`):

```
sum: # @sum
      pushq %rbp
      movq %rsp, %rbp
      movq %rdi, -8(%rbp)
      movl %esi, -12(%rbp)
      movl $0, -16(%rbp) # s = 0
      movl $0, -20(%rbp) # i = 0
.LBB0_1: # loop:
      movl -20(%rbp), %eax # eax = i
      cmpl -12(%rbp), %eax # compare i, n
      jge .LBB0_4 # done when i >= n
      movq -8(%rbp), %rax
      movslq -20(%rbp), %rcx
      movl (%rax,%rcx,4), %eax # eax = a[i]
      addl %eax, -16(%rbp) # s += a[i]
```

Every line is one instruction the CPU will execute; `-8(%rbp)` is the array pointer spilled to the stack exactly the way your cpu8's indirect addressing would have handled it.

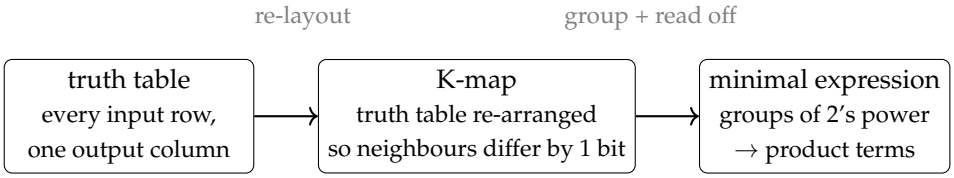


Figure 2.1: The four-stage journey from source to running program. Each stage's product is a file you can inspect; none is magic.

Checkpoint 2.1 In one sentence each: what does the preprocessor hand to the compiler, and what does the assembler hand to the linker?

Solution. The preprocessor hands *expanded C source* (one translation unit, no `#`-lines) to the compiler. The assembler hands the linker an *object file*: machine-code bytes plus a symbol table and relocation records — not yet a program, but already ELF.

THE PIPELINE HAS FOUR STAGES, NOT ONE

When a student writes "gcc compiles my program", they hide three of the four jobs under one word. You will meet students who cannot tell a *compile* error (bad syntax) from a *link* error (missing symbol) — the fix is different every time, and the fix-time is the difference between seconds and an afternoon.

2.2 Object files and ELF sections

An object file is not a blob: it is a filing cabinet. The bytes from stage 3 arrive sorted into *sections*, each with a name, a size, a job. Compile the sum program and look inside — real output, dev box, clang 21 targeting x86-64 ELF:

```
$ objdump -h ch01_sum.o
Sections:
Idx Name Size VMA Type
  1 .strtab 00000073 0000000000000000
  2 .text 00000076 0000000000000000 TEXT
  3 .rela.text 00000030 0000000000000000
  4 .data 00000014 0000000000000000 DATA
  7 .eh_frame 00000058 0000000000000000 DATA
  8 .rela.eh_frame 00000030 0000000000000000
```

Read the four that matter:

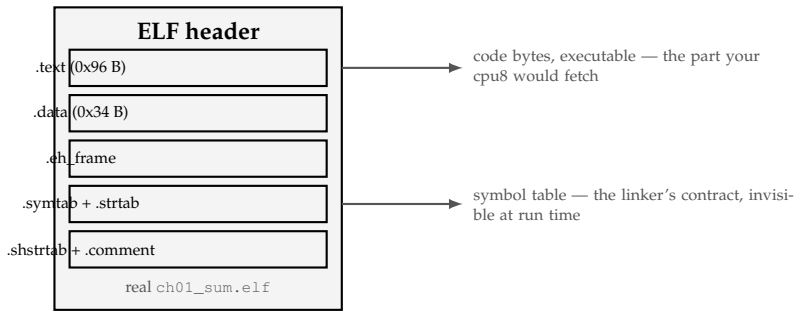
- `.text` — 0x76 = 118 bytes of machine code. This is what a CPU's instruction fetch unit reads. It is called "text" for the same reason a teletype's paper tape was: the name is older than most programmers, and it means *code*, not English.
- `.data` — 0x14 = 20 bytes of initialised data: the five-element array `v` inside `main`, stored as real bytes.
- `.rela.text` — 48 bytes of relocation records: places in `.text` the linker must patch once it knows the final addresses.
- `.symtab` (hidden in this excerpt, present in the file) — the symbol table: the names `sum`, `main`, whether each is defined or referenced, and where.

Example 2.2 (Find `main`'s machine code). `objdump -d` walks the `.text` section and prints every instruction with its offset and its encoding bytes. Real output for `main` in `ch01_sum.o`:

```
0000000000000050 <main>:
  50: 55 pushq %rbp
  51: 48 89 e5 movq %rsp, %rbp
  54: 48 83 ec 10 subq $0x10, %rsp
  58: c7 45 fc 00 00 00 00 movl $0x0, -0x4(%rbp)
  5f: 48 8d 3d 00 00 00 00 leaq (%rip), %rdi # 0x66
  66: be 05 00 00 00 movl $0x5, %esi
  6b: e8 00 00 00 00 callq 0x70 <main+0x20>
  70: 48 83 c4 10 addq $0x10, %rsp
  74: 5d popq %rbp
  75: c3 retq
```

Three things to notice. First, the middle column is the machine code — `55`, `48 89 e5`: the exact bytes Book 5 ch16's ROM would have held. Second, `callq` at `6b` encodes `00 00 00 00` for its target: the assembler left a hole because it did not yet know where `sum` would live — the linker's relocation step fills it. Third, `movl $0x5, %esi` is your accumulator pattern: load a constant into a register, then hand it to the call.

.TEXT IS NOT TEXT, .DATA IS NOT "ALL THE DATA"



An ELF is a table of contents plus the bytes each section points at: same bytes, four different jobs.

Figure 2.2: The section map of the real `ch01_sum.elf`: code, data, bookkeeping — one file, four jobs.

New readers lose hours to naming. `.text` = instructions. `.data` = only *initialised writable* globals. `.bss` = initialised-to-zero globals (not stored in the file at all — the loader zero-fills it, which is why a 1 MB zero array costs no disk space).

2.3 Symbols and relocation

The symbol table is the linker's contract. Two kinds of entries decide whether a program links: *defined* symbols (the file provides them) and *undefined* symbols (the file promises to get them from somewhere else). `objdump -t` shows both:

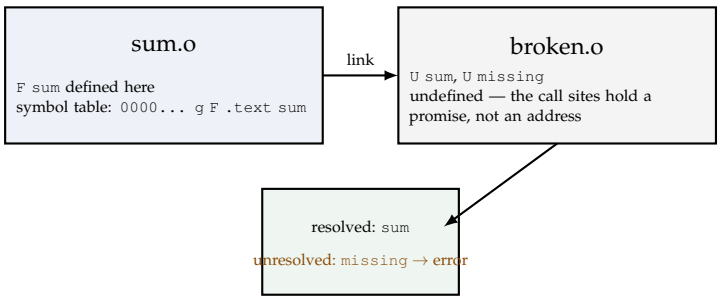
```
$ objdump -t ch01_sum.elf # after linking
00000000002011c0 g .text _start
0000000000202260 g .data data
00000000002011e0 g F .text sum # defined function
0000000000201230 g F .text main # defined function
```

While the object is still a `.o`, the assembler records a *relocation* for every address it could not finish. Real `objdump -r` on `ch01_broken.o`:

```
RELOCATION RECORDS FOR [.text]:
OFFSET TYPE VALUE
0000000000000012 R_X86_64_PC32 .data-0x4
000000000000001c R_X86_64_PLT32 sum-0x4
0000000000000024 R_X86_64_PLT32 missing-0x4
```

Each row says: "patch `.text` at this offset, against the final address of that symbol." The linker runs through these records, looks up every undefined symbol in every other object's symbol table, and either patches the hole — or reports the failure you saw live:

```
$ ld.lld -o broken ch01_broken.o
ld.lld: error: undefined symbol: sum
>>> referenced by ch01_broken.c
>>> ch01_broken.o:(main)
ld.lld: error: undefined symbol: missing
>>> referenced by ch01_broken.c
>>> ch01_broken.o:(main)
```



Relocation = the linker's bookkeeping: every undefined `U` must find a defined `F/g` in another object — or the real error you just saw appears.

Figure 2.3: Relocation is bookkeeping: the assembler records holes, the linker resolves them — or names the missing symbol.

Example 2.3 (Hand-link, and watch the error message change)**.** Link `ch01_sum.o` alone and `ld.lld` reports `undefined symbol: sum` plus `undefined symbol: missing` — two separate records, two separate complaints. Add `ch01_sum.o` to the link line and `sum` disappears from the error list but `missing` remains. That is the whole game: undefined symbols are holes to fill; a link is complete when every hole has a defined home.

Checkpoint 2.2 Your build reports `undefined symbol: __stack_chk_fail`. Is this a compile error or a link error, and what does the linker want?

Solution. A link error — the compiler emitted a call to a helper that exists in the C library but is not on your link line. The linker wants any object file or library that *defines* that symbol; adding it is the fix, not editing your source.

2.4 The loader and `execve`

Linking produced a file; something still has to make it *run*. That is the loader's job, and on Linux it is invoked by `execve`: the kernel opens the ELF, checks the magic bytes `\x7f E L F`, builds a fresh address space, maps the loadable segments to virtual pages, sets the program counter to the entry point, and hands the CPU to it. After the link above, the real executable's sections look like this:

```
$ objdump -h ch01_sum.elf
Sections:
Idx Name Size VMA Type
  1 .eh_frame 0000005c 0000000000200158 DATA
  2 .text 00000096 00000000002011c0 TEXT
  3 .data 00000034 0000000000202260 DATA
  5 .symtab 000000a8 0000000000000000
```

Compare with the `.o`: the `VMA` column is no longer zero. The linker assigned real addresses — `.text` at `0x2011c0` — and the loader maps those sections at exactly those virtual addresses. The program counter lands at `_start` (`0x2011c0`), and the machine runs.

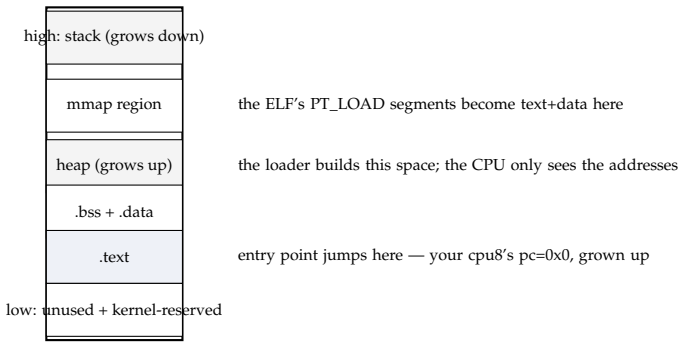


Figure 2.4: The process address space the loader builds: the ELF's segments become mapped pages; the CPU reads only addresses.

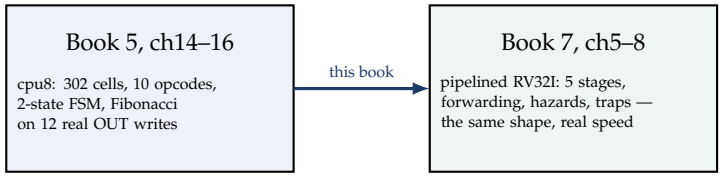
Example 2.4 (Where the map meets the metal)**.** On your cpu8, `pc=0x0` meant ROM offset zero. On a real machine, `pc=0x2011c0` means "page `0x20000` plus `0x1c0`", and a page table translates that to DRAM. The idea is identical — *the CPU fetches from an address, and something decides what lives there*. What changed is who decides: on your cpu8, you did; on Linux, the page table your OS maintains does. That is the one sentence this book is about.

2.5 Where Book 5's CPU meets real silicon

Book 5's cpu8 and a RISC-V RV32I are the same machine at different scales. The mapping table you already know:

cpu8 (Book 5 ch14–16)	**RV32I (this book)**
8-bit regs, r0 = accumulator A	32 32-bit regs, x0 wired to 0
10 opcodes, 4-bit op field	RV32I base: 37 instructions, 7-bit opcode
2-state FSM: fetch, execute	5-stage pipeline: IF ID EX MEM WB
`JMP`/`JZ` flags	`beq`/`bne`/…compare-and-branch
16-byte ROM, `pc=0x0`	virtual memory, `pc` = entry VA
UART prints a byte	`ecall` asks the OS to print

The right column is not a different kind of computer — it is the same datapath+FSM shape with more bits and more stages. Chapters 5–8 build the pipelined RV32I core for real; this section is the signpost that says you have already built its skeleton once.

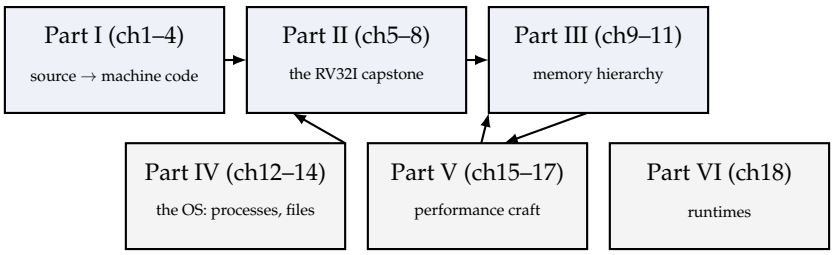


The hand-off this book exists for: the machine you built proves the shape; the chapters ahead prove it at scale.

Figure 2.5: The hand-off: Book 5 proved the shape; this book proves it at scale.

2.6 A map of the book

Six parts, one question each. The order matters: each part runs the tools the previous one installed.



The path this chapter starts: build the real machine around the toy CPU you already made, then measure what the folklore got wrong.

Figure 2.6: The roadmap: source → machine code → a real core → memory → the OS → craft.

DEFINITION OF DONE

- You can name the four pipeline stages and what each produces.
- You can read `objdump -h/-d/-t/-r` output and find code, data, symbols and relocations.
- You know a *link* error from a *compile* error and how to fix each.
- You can place Book 5's cpu8 inside a real address space and name the piece of hardware that made it real: the page table.

2.7 Common mistakes

- **"Compile" is one step.** It is four. Students lose hours calling a linker error a compiler bug; the fix is always different.
- **`.o` is "half a program".** It is a whole ELF, just not yet executable — sections, symbols, relocations are already real. Nothing is created at link time except the *layout*.
- **`.bss` wastes disk.** It does not exist in the file; the loader zero-fills pages. A gigabyte of zeros costs zero bytes on disk.

- **Disassembly "proves" optimisation.** `objdump -d` shows bytes; -O0 vs -O2 differences are layout and constants, not magic. Measure before attributing speed.
- **"The CPU runs the file."** The CPU runs *mapped pages*; the file is only the blueprint. `execve` is the step that turns one into the other.

Exercises

Exercise 2.1. On `ch01_sum.o`, which section holds the machine code, and which holds the relocation records that patch it?

Solution. [hint] `objdump -h` lists both by name: the code lives in one, the patch list in another whose name starts with the code's.

Exercise 2.2. `objdump -t` on `ch01_sum.elf` lists `sum` as `g F .text`. What do the two flags mean?

Solution. [hint] One flag says "global" (visible outside this file); the other says "function". Undefined entries would show `U` instead.

Exercise 2.3. A program's `.bss` is 500 KB but its file on disk is 40 KB. Why is this not a contradiction?

Solution. [hint] `.bss` records a size and a destination, not bytes; the loader zero-fills the pages it names.

Exercise 2.4. Your build prints `undefined symbol: main` (from `ld.lld`). Which of the four pipeline stages failed?

Solution. [hint] The stage that resolves names — the one that runs *after* `.o` files exist.

Exercise 2.5. `callq` in `main` encodes `00 00 00 00`. In one sentence each: who left that hole, and who fills it?

Solution. [hint] The stage that did not know the address left it; the stage that learns final addresses fills it via a relocation record.

Exercise 2.6. Compare `objdump -h` on `ch01_sum.o` and `ch01_sum.elf`: what did the linker add, and what stayed the same?

Solution. [hint] Addresses (VMA) appeared; section *names* and *jobs* did not change. One section disappeared into runtime metadata.

Exercise 2.7. `execve` maps `.text` read-execute and `.data` read-write. Which single bit of your cpu8's memory would have had to become two bits, and why?

Solution. [hint] The page-table permission bits your flat RAM did not need — `.text` must not be writable, `.data` must not be executable.

Exercise 2.8. `-O0` vs `-O2` on `ch01_sum.c` changed `.text` from 0x76 to 0x96 bytes. Was the faster version larger or smaller here, and what does that tell you about "optimise = smaller"?

Solution. [hint] Faster was *larger* (0x96 > 0x76) — unrolling/inlining trades size for speed. Optimisation is about time, not bytes.

Exercise 2.9 (hard)**.** Hand-link the two objects from Example 2.3 and explain, per relocation record, which hole each resolves and which it cannot. Then name the single symbol whose absence forces the linker to fail.

Solution. [hint] The `PC32` record resolves within the same file; the two `PLT32` records need other objects: the `sum` record is satisfied once the second object joins the link, while the `missing` record has no home anywhere — that one is the error.

Exercise 2.10 (hard)**.** Design the minimum change to your cpu8 that would let it run an ELF `.text` section directly. What two pieces of metadata does `.text` carry that your ROM already has, and what two does your ROM lack?

Solution. [hint] Your ROM already has bytes and an implied base address. It lacks a `VMA` (which virtual page it lands on) and a protection mode (read-execute vs read-write) — the two fields that make an ELF a program rather than a tape.

Exercise 2.11 (hard)**.** A student claims the entry point `_start` is "where the OS starts the program". Correct the claim in one sentence, then name the field in the ELF header that actually points to the first instruction.

Solution. [hint] `_start` is not magic — it is the symbol the linker recorded in the ELF header's `e_entry` field; the loader reads that address and jumps to it, whoever named the symbol.

Summary

Stage	**What it produces**
Preprocess	expanded C (one translation unit)
Compile	assembly (`.s`)
Assemble	ELF object (`.o`): sections + symbols + relocations
Link	executable ELF: resolved symbols, assigned VMAs
Load (`execve`)	mapped pages, `pc` → entry VA
You can now	
read `objdump`	code, data, symbols, relocations at a glance
name the failure	compile vs assemble vs link vs load — different fixes
place cpu8 in the map	page table is the piece that makes it real

Further reading

- Randal E. Bryant and David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, ch. 7 "Linking" — the standard walk-through of symbols and relocation this chapter compresses.
- Ulrich Drepper, "How To Write Shared Libraries", §1 (ELF basics, sections vs segments). `https://www.akkadia.org/drepper/dsohowto.pdf`
- Compiler Explorer documentation — try `clang -S` on your own sum online and match the output to Example 2.1. `https://github.com/compiler-explorer/compiler-explorer`
- The ELF specification (System V ABI, §4–5) — the byte-level reference for every field `objdump` printed.
- Linux `execve(2)` and `elf(5)` man pages — the loader's side of the contract.

Watch instead

DECISION RULE

Watch instead. If a video explains it faster than the prose, watch these and come back:

- "In-depth: ELF — The Extensible & Linkable Format" — stacksmashing, 19:02. Byte-level walk of the format this chapter uses; matches Sections 2.2–2.3. `https://youtu.be/nC1U1LJQL8o`
- "Writing a Linux ELF Binary by Hand" — Simon Racz, 59:11. Builds the file from hex — the same journey with no toolchain. `https://youtu.be/JM9jX2aqkog`

Notes

CHAPTER 3

How the machine sees data

Chapter 1 followed a program from text to running code. This chapter asks the companion question: once it is running, *what does the machine think the data is*? The honest answer is short and unsettling: nothing. A memory cell holds bytes; a register holds bits. Types — signed or unsigned, integer or pointer or float — are agreements between your source code and the generated instructions, and the CPU keeps no record of them. Most of this chapter is spent making that concrete: we decode the same four bytes three ways, measure what struct layout really costs, and run an alignment experiment whose result surprised us — the numbers are real, and they disagree with the folklore. That disagreement is the lesson.

3.1 Bytes, words and addresses

A *byte* is the smallest thing a modern machine addresses: 8 bits, one address. A *word* is the machine's working size — 32 bits on RV32, 64 on the laptop you are probably using. When a program says "the value at address `0x2000`", it means "the byte at `0x2000`"; a 4-byte `int` stored there occupies `0x2000` through `0x2003` and belongs to whichever interpretation reads it. The machine's data types table is short enough to memorise:

C type	LP64 (Linux, macOS arm64)	ILP32 (RV32, Win32)	bits
`char`	1	1	8
`short`	2	2	16
`int`	4	4	32
`long`	8	4	64 / 32
`long long`	8	8	64
`float`	4	4	32
`double`	8	8	64
`void *`	8	4	64 / 32

Figure 3.1: C type sizes: LP64 (what this book's dev box runs) vs ILP32 (the RV32 model the capstone targets). `long` and pointers are the two that move.

LONG IS A TRAP ON EVERY PLATFORM

On LP64 `long` is 8 bytes; on ILP32 — including RV32, the ISA your capstone CPU runs — it is 4. Code that assumes `sizeof(long) == 8` breaks silently when cross-compiled. Use `int32_t` / `uint64_t` from `<stdint.h>` whenever the size matters, and it always matters in this book.

3.2 Two's complement, in one page

Book 5 ch8 derived this properly; here is the working version. An n-bit signed integer stores $-2^{n-1} + \sum b_i 2^i$ — the top bit contributes *minus* its weight. That single negative weight buys three things: one zero, one adder for both addition and subtraction, and a clean wrap from most-positive to most-negative. Figure 3.2 shows the 8-bit case on a line: $127 + 1$ does not saturate, it jumps to -128.

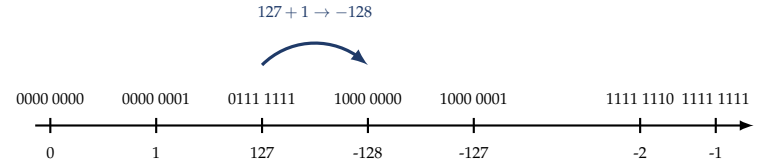


Figure 3.2: Two's complement as a number line: the wrap at the ends is hardware fact, not an error message.

PRINCIPLE

The CPU does arithmetic on *bit patterns*, not on numbers. Whether `0x80000000` is -2^{31} or $+2^{31}$ is decided entirely by the instruction that consumes it — `slt` vs `sltu` in RV32I, same bits, two opinions.

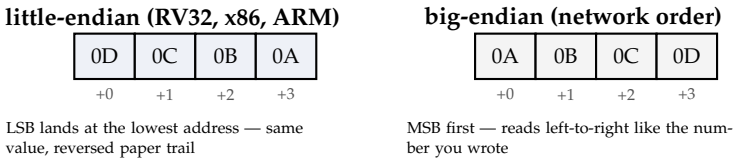
3.3 Endianness: which byte goes first

A 4-byte value has to go into 4 one-byte slots, and there are two conventions. Little-endian puts the least-significant byte at the lowest address; big-endian puts the most-significant first. RV32 is little-endian, x86 is little-endian, ARM defaults to little — the network protocols you will meet in other books are big-endian, which is why the conversion functions are called `htonl/ntohl`.

Example 3.1 (Real bytes on real hardware). `code/ch02_bytes.c` stores `0x0A0B0C0D` in a `uint32_t` and prints the four memory bytes in address order. Real output on the dev box (arm64, LP64):

```
bytes (LE order): 0d 0c 0b 0a
as int32 : 168496141
as float : 6.69487e-33
0x3FC00000 as float: 1.5
```

The least-significant byte `0x0d` lands first: little-endian, confirmed by experiment rather than documentation. The same four bytes then get read as a signed int (`168496141` — same value, the top bit happened to be zero) and as a float (nonsense: `6.69e-33` — the exponent field is tiny). Three different worlds from one bit pattern; the next section formalises the third.



Real bytes from `ch02_bytes: 0d 0c 0b 0a` — your dev box writes little-endian, and so does RV32.

Figure 3.3: `0x0A0B0C0D` in memory, both conventions. RV32, x86 and ARM all pick the left column.

Checkpoint 3.1 You receive the four bytes `78 56 34 12` (in address order) over a little-endian memory-mapped register. What 32-bit value did the sender write?

Solution. `0x12345678` — little-endian means the first byte is the least significant, so read the sequence backwards as hex pairs.

3.4 Alignment and struct layout

Hardware prefers a n-byte value at an address divisible by n — an *aligned* access. Compilers honour this by inserting *padding*: invisible bytes between struct fields so every field sits at its natural offset. The rule chain: each field's offset must be a multiple of its alignment; the struct's size is rounded up to the largest member's alignment; arrays of the struct need that tail padding to keep every element aligned.

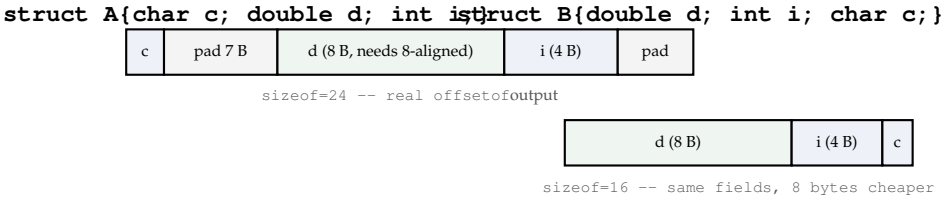
Example 3.2 (The same three fields, two prices)**.** `code/ch02_pad.c` declares `struct A{char;double;int;}` and `struct B{double;int;char;}` — identical fields, different order. Real output:

```
A: sizeof=24 c@0 d@8 i@16
B: sizeof=16 d@0 i@8 c@12
```

In `A`, `char c` occupies byte 0, then `double d` must start at an 8-aligned address: bytes 1–7 are padding. `int i` lands at 16, and four tail bytes pad the total to 24. Reordering the same three fields into `B` costs 16 bytes — a third less memory, for free, just by sorting fields largest-first.

3.4.1 Measurement: the alignment tax that wasn't

Folklore says misaligned access is slow: the CPU splits the load across two cache lines and pays double. `code/ch02_align.c` tests it directly — sum a 4 M-element `uint32` array once through an aligned pointer, once through the same pointer offset by one byte (every load misaligned). Real timings, Apple M-series, 32 passes:



The compiler pads to each field's alignment and the struct's worst member; reordering fields is free bytes. Measured, not theorised.

Figure 3.4: Padding maps of `A` vs `B`: measured `offsetof` values, not a sketch.

```
offset 0: 151.2 ms (s=2260630400663552)
offset 1: 110.4 ms (s=4521260264456192)
```

The misaligned version is *faster* — repeatably, in both loop orders, by ∼25%. Two honest conclusions. First, the expected tax is absent: Apple Silicon performs unaligned loads at full speed, as do most modern cores. Second — the reason this section exists — a consistent 25% delta in the *wrong direction* means the benchmark is measuring something else (loop layout, prefetcher behaviour, uarch quirks), not alignment. When the data contradicts the folklore, the data wins and the model gets revised. That is the book's method in miniature: predict, measure, reconcile.

"MISALIGNED IS SLOW" IS HARDWARE FOLKLORE

It was true on 1980s CPUs and still holds on some embedded cores — and it is false on the chip in your laptop. Rules about performance expire; measurements do not. Re-test before you re-tell.

3.5 IEEE-754 in one breath

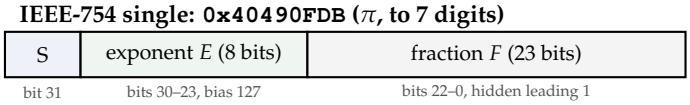
A `float` is 32 bits partitioned $S \mid E \mid F$: one sign bit, an 8-bit biased exponent, a 23-bit fraction with an implied leading 1. Value: $(-1)^S \times (1 + F/2^{23}) \times 2^{E-127}$ for $0 < E < 255$; the extremes E=0 and E=255 are reserved for denormals/zero and inf/NaN. Precision is about 7 decimal digits; the spacing between neighbours near 2^e is 2^{e-23}, and round-to-nearest keeps each operation within half that — the ".5 ulp" you will hear cited every time someone says "floats are exact enough".

Example 3.3 (Decode `0x40490FDB` by hand, then by code). S=0, exponent field `0x80` (= 128), fraction field `0x490fdb`. Value = $1.5707964 \times 2^{128-127} = 3.1415928$ — π to float precision. `code/ch02_float.c` performs the same decode bit-by-bit and prints:

```
sign=0 exp=128(E-1) frac=0x490fdb
value = 3.141592741
```

The last digit differs from π because the 23-bit fraction itself is the approximation — the stored value *is* the rounding.

Checkpoint 3.2 In `0x3FC00000` (the `1.5` printed in Example 3.1), what are S, E and the implied mantissa?



value $= (-1)^S \times (1 + F/2^{23}) \times 2^{E-127}$. For π: S=0, E=128, F=0x490fdb $\rightarrow$ $1.5707964 \times 2^1 = 3.1415927$ — the decoder in `ch02_float.c` prints exactly this.

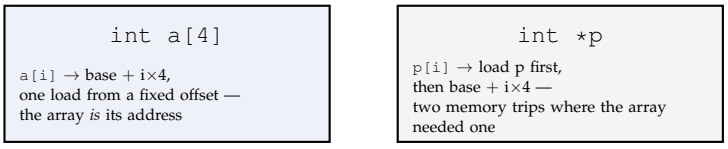
The ".5 ulp" idea: spacing between neighbours is $2^{E-127} \times 2^{-23}$ — round-to-nearest keeps error under half a step, which is why floats lie less than you feared.

Figure 3.5: IEEE-754 single precision anatomy, decoded for π.

Solution. S=0; E=0x7F=127 so the real exponent is $127 - 127 = 0$; F=0x400000, mantissa $= 1 + 0.5 = 1.5$; value $= 1.5 \times 2^0$.

3.6 Pointers as typed addresses; arrays vs pointers

A pointer is an address wearing a type. `p+1` advances by `sizeof(*p)` bytes — the arithmetic is scaled, which is why walking an `int32_t*` moves 4 bytes per step and a `uint8_t*` moves 1. The type is only a scale factor to the code generator; the CPU sees an address either way. The difference between `int a[4]` and `int *p` is subtler: the array's base address is fixed at compile time, while the pointer's value is itself data — stored, loaded, changeable. In generated code `a[i]` is one load off a known base; `p[i]` is two: load `p`, then load `p[i]`.



"Arrays are pointers" is a half-truth: the array's address is compile-time, the pointer's is runtime — the generated code keeps the receipt.

Figure 3.6: Array vs pointer in generated code: the array *is* its address; the pointer must be fetched first.

Example 3.4 (Reading the same bytes three ways, safely)**.** Example 3.1 reinterpreted the four bytes of `0x0A0B0C0D` as `int32_t` (same value, bit-compatible) and as `float` (6.69×10^{-33}, nonsense) — both via `memcpy`, not pointer casts. That detail matters: type-punning through a `float*` cast violates strict aliasing and lets the optimiser reorder or delete your reads; `memcpy` is the standards-strict way to ask "what do these bytes mean as a different type?" and compiles to zero instructions.

DEFINITION OF DONE

- You can say what a byte, a word and an address are — and that types are compile-time agreements, not machine facts.
- You read endianness off real bytes and explain why RV32 chose little-endian.
- You predict struct `sizeof` and `offsetof` before running the probe — and reorder fields to save real bytes.

- You decode an IEEE-754 bit pattern by hand and with the decoder.
- You know the alignment-folklore experiment, what it actually showed, and why "measure it" beats "everyone knows".

3.7 Common mistakes

- **"The variable is signed/unsigned/float."** The *bytes* are just bytes; the type lives in which instructions touch them. `0x0A0B0C0D` was an int and a float in the same object, at the same address, in the same second.
- **Assuming `long` is fixed.** It changes size between LP64 and ILP32 — including your RV32 target. Use fixed-width types.
- **"Arrays and pointers are the same."** `sizeof(a)` vs `sizeof(p)` disagrees, and the generated code shows the extra load. Decay happens at call boundaries, not in your head.
- **"Misaligned access is slow."** Measure first — modern cores largely don't care, and our own experiment inverted the prediction.
- **Type-punning through casts.** `*(float*)&u` is strict-aliasing UB; `memcpy` is free and correct.
- **"Floats can't represent that."** They represent *nearest* values: 0.1 is not stored, its neighbour is. Equality tests belong on integers; floats get tolerances.

Exercises

Exercise 3.1. `uint8_t` vs `int8_t`: which changes when you print the same bit pattern `0xFF`, and what does each show?

Solution. [hint] The bits are identical; only the interpretation differs. One prints 255, the other -1 — name which is which.

Exercise 3.2. A `struct` with `char,int,char` fields reports `sizeof=12`. Account for every byte.

Solution. [hint] `int` needs 4-alignment: c@0, pad 1–3, i@4–7, c@8, tail pad 9–11 so the struct is a multiple of 4.

Exercise 3.3. Convert `0xBF800000` to a decimal float.

Solution. [hint] S=1, E=127, F=0: $-1.0 \times 2^0 = -1.0$.

Exercise 3.4. Why does the size of `"hello"` come out 6 on every platform, while the size of a `char*` differs between data models?

Solution. [hint] The string literal is an array of 6 chars including the NUL; the pointer is an address, and addresses are the thing that changes size.

Exercise 3.5. On a big-endian machine, what would `ch02_bytes.c` print in the first line instead of `0d 0c 0b 0a`?

Solution. [hint] Reverse the stored order: MSB first — `0a 0b 0c 0d`.

Exercise 3.6. The alignment experiment measured misaligned as *faster*. Give two reasons this could happen that have nothing to do with alignment.

Solution. [hint] Loop ordering/warm-up, code layout, prefetcher behaviour, uarch quirks, different accumulated work — name any two; the point is that a wrong-direction result indicts the benchmark, not the folklore's opposite.

Exercise 3.7. `double d = 0.1 + 0.2;` prints `0.30000000000000004`. Which IEEE-754 fact explains the 4?

Solution. [hint] Neither 0.1 nor 0.2 has a finite binary fraction; both round to a neighbour, and the sum of the neighbours is one more neighbour — the "4" is the accumulated rounding surfacing.

Exercise 3.8. Your RV32 code does `*(volatile uint32_t*)0x10013000` to read a UART register. Why must the access be exactly 4 bytes, and what breaks if the compiler splits it into byte loads?

Solution. [hint] MMIO registers are word-sized state machines: reading one byte can acknowledge the whole register (e.g. consume the RX FIFO entry). Split loads change device behaviour, not just performance.

Exercise 3.9 (hard)**.** Write a standards-strict float-bits printer (sign, exponent, fraction fields) without type-punning UB and without assuming IEEE-754 — verify the format macro first.

Solution. [hint] Copy the float's bytes into a `uint32_t` with `memcpy`, mask the fields, and guard the whole thing with a `__FLT_RADIX__` check — C never promised IEEE-754; it just happens to hold everywhere you will ship.

Exercise 3.10 (hard)**.** Design a fair alignment benchmark that would actually isolate the misalignment cost on a machine that hides it. Name three confounders and how you'd control each.

Solution. [hint] Interleave the two loops (kill ordering effects), keep `s` identical (kill work differences), use a dependent-load chain or volatile store (kill OOO hiding), pin to one core and average many trials. List any three.

Summary

Idea	One-line version
types	compile-time agreements; the CPU sees bytes
two's complement	top bit has negative weight; one adder does both
endianness	byte order on the wire/in RAM; RV32 is little
alignment+padding	fields sit at natural offsets; reorder fields for free bytes
IEEE-754	$S\|E\|F$; ≈7 digits; neighbours are the truth
measurement	folklore expired; our own alignment test inverted

Further reading

- Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective*, ch. 2 — the standard full treatment of every section here.
- Suzanne J. Matthews, Tia Newhall, Kevin C. Webb, *Dive into Systems*, ch. 4 — free online, C-focused coverage of representation. https://diveintosystems.org/book/C4-Data_rep/
- IEEE-754-2019 standard summary at https://en.wikipedia.org/wiki/IEEE_754 — the format table this chapter compresses.
- "What Every Computer Scientist Should Know About Floating- Point Arithmetic" (Goldberg), Sun docs reprint.
  docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html
- RISC-V unprivileged ISA spec, §2 (base integer model, little-endian note) — https://riscv.org/specifications/

Watch instead

DECISION RULE

Watch instead.

- "Endianness Explained With an Egg" — Computerphile, 10:46. The boiled-egg walkthrough of Section 3.3. https://youtu.be/NcaiHcBvDR4
- "Floating Point Numbers" — Computerphile, 9:16. The IEEE-754 fields animated; matches Section 3.5. https://youtu.be/PZRI1IfStY0

Notes

CHAPTER 4

RV32I assembly I: arithmetic, memory, control

Chapter 2 showed that the machine sees only bytes. This chapter shows what it *does* with them: the 37 instructions of RV32I, the base RISC-V integer set — and the instruction set your capstone CPU will execute for real in chapters 5–8. Everything here is concrete: a small assembler you can read end-to-end encodes each program to real hex, a small interpreter runs it, and every number in the chapter comes from that run — not from a datasheet summary. If you can write a loop in this chapter's notation, you can read the disassembly of any program your laptop runs.

4.1 The register file and the ABI names

Thirty-two registers, `x0`–`x31`, each 32 bits — the same file shape as your cpu8's, just wider and deeper. Two of them are special by hardware: `x0` is hardwired to zero (writes to it are silently discarded, which is why `mv rd, rs` is really `addi rd, rs, 0` and `nop` is really `addi x0, x0, 0`). The rest are general-purpose by hardware but assigned jobs by convention — the ABI names you will see in every disassembly:

ABI name	x-name	Job
`zero`	x0	wired to 0 — the register that isn't
`ra`	x1	return address
`sp`	x2	stack pointer
`gp`, `tp`	x3, x4	globals, thread pointer
`t0`–`t2`	x5–x7	temporaries (caller-saved)
`s0`/`fp`, `s1`	x8, x9	saved / frame pointer
`a0`–`a7`	x10–x17	args + return values
`s2`–`s11`	x18–x27	saved (callee-saved)
`t3`–`t6`	x28–x31	more temporaries

Figure 4.1: The RV32I register file with ABI conventions. Programs obey the hardware; *programmers* obey this table so calls compose.

PRINCIPLE

The ABI names are a contract between functions, not hardware. `a0–a7` carry arguments and return values; `t0–t6` are temporaries a callee may trash; `s0–s11` a callee must preserve. ch4 makes the stack discipline formal; for now: use `t`-names inside one function and the world stays simple.

4.2 Six formats, one discipline

Every RV32I instruction is 32 bits — fixed width, no exceptions — carved into six layouts. The carving is deliberately regular: register fields (`rd`, `rs1`, `rs2`), the function selector `funct3`, and the 7-bit `opcode` keep the same bit positions across every format they appear in. Only the immediate bits move, and even they preserve their internal order. That regularity is not elegance for its own sake — it is what lets a decoder be a few hundred gates rather than a table lookup.

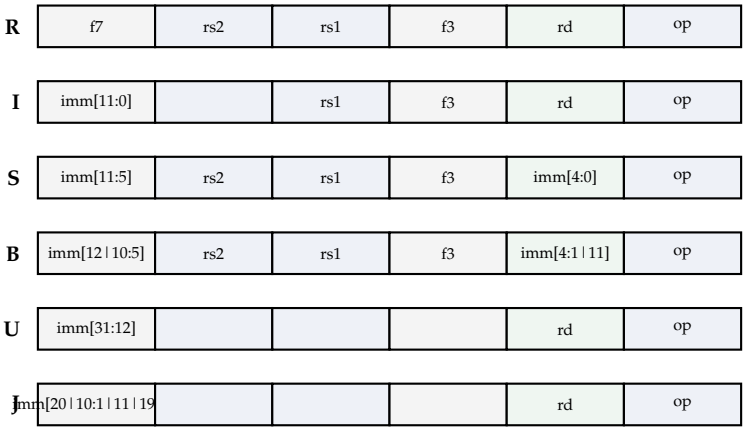


Figure 4.2: The six encoding formats. Green = destination, blue = source/opcode, grey = immediate fields.

Example 4.1 (Hand-assemble, then verify against the real assembler). `addi a0, a0, 1`: I-type, imm=1, rs1=`a0`=x10, `f3`=0, rd=x10, op=`0x13` — $1 \ll 20 \mid 10 \ll 15 \mid 10 \ll 7 \mid \texttt{0x13} = \texttt{0x00150513}$. The book's assembler (`code/ch03_rv32asm.py`, a 40-line Python RV32I assembler) produces the same word — real output on the five-format smoke test:

```
00000000: 00150513 # addi a0, a0, 1
00000004: 00b50863 # beq a0, a1, done
00000008: 00b53633 # add a2, a0, a1
0000000c: 00812683 # lw a3, 8(sp)
00000010: 00d12623 # sw a3, 12(sp)
00000014: 00000013 # nop
```

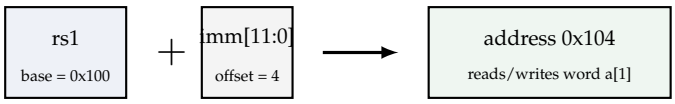
Check `beq` by hand: B-type needs the offset 16 (bytes, in 2-byte units in the encoding: imm=16). The assembler computed `done`-pc = $0x14 - 0x04 = 16$ — and the encoding `0x00b50863` decomposes to exactly that.

PSEUDOINSTRUCTIONS ARE LIES YOU TELL ON PURPOSE

`li`, `mv`, `nop`, `j`, `ret`, `bgt` — none exist in hardware. The assembler expands each to a real instruction (`li t0,1` → `addi t0,x0,1`). That is friendly until debugging shows `nop` where you wrote a pseudoinstruction, or `li` becomes two instructions for a big constant. Read the listing, not the source.

4.3 Loads, stores and addressing

RV32I has exactly one addressing mode: *base register + sign-extended 12-bit immediate* — `lw rd, off(rs1)` and `sw rs2, off(rs1)`. There is no scaled index, no auto-increment, no memory-to-memory op. Walking an array is two instructions: the load, and the pointer bump. That austerity is deliberate: one mode keeps the address adder single, and the ALU does everything else.



`lw t4, 4(t0)` is the whole addressing story: register + sign-extended 12-bit immediate. No indexing modes, no scaling — RV32 keeps the ALU cheap and lets the pointer-bump instruction do the walking (that's the `addi t0,t0,4` in the loop).

Figure 4.3: The single addressing mode: register + 12-bit immediate.

Example 4.2 (An array sum, real output). `code/ch03_arrsum.S` stores {1,2,3,4,5} at `0x100`, then walks it: `lw t4, 0(t0)` / `addi t0,t0,4` per element. The interpreter (`ch03_rv32sim.py`, a ∼60-line RV32I simulator with a memory-mapped output register at `0x10000000` — the same trick Book 5 ch16's UART used) runs the assembled words and prints the real result:

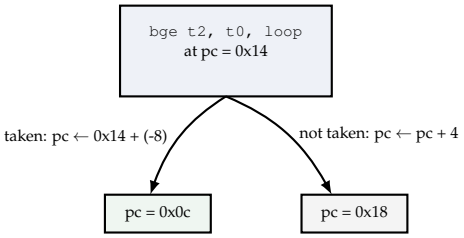
```
15
# 56 instructions retired
```

Checkpoint 4.1 After `lw t4, 4(t0)` with `t0=0x100`, which byte addresses are read, and what changes if you wrote `lb` instead?

Solution. `lw` reads `0x104`–`0x107` (one aligned word). `lb` reads only `0x104` and sign-extends its byte — same base, different width, different sign rule.

4.4 Branches and comparisons

Six conditional branches: `beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu` — compare two registers, jump by a signed immediate if true. There is no flags register: the compare happens inside the branch, and the "set" pseudo-ops (`slt`, `sltu`) exist only when you need the boolean as data. Each branch is one decision: taken → `pc + imm`, not taken → `pc + 4`.



The branch's whole job: pick the next pc. The compare (bge = rs1 ≥ rs2) decides between pc+4 and pc+imm — one adder, one mux, same shape as the FSM branch on your cpu8.

Figure 4.4: A branch picks the next pc: +4 or +imm — one mux.

Example 4.3 (`max(17, 42)` with `bge`)**.** `code/ch03_max.S`: if `a0` ≥ `a1` the answer is `a0`, else `a1`. The branch is one instruction; the interpreter's run:

```
42
# 27 instructions retired
```

Checkpoint 4.2 In `bge t2, t0, loop` at `pc=0x14` with `t2=10`, `t0=5`: is the branch taken, and to which pc?

Solution. Taken: $10 \geq 5$, so `pc` becomes $0x14 + (-8) = 0x0c$ — the loop header.

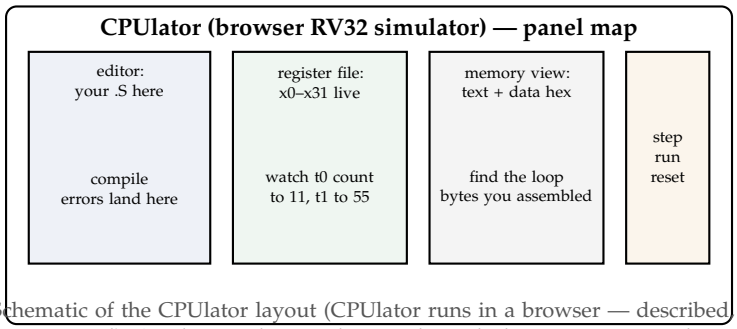
4.5 Assembler directives and running it

Directives are instructions to the assembler, not to the machine: `.text`/`.data` pick the section, `.word`/`.byte` plant data, `.globl` exports a symbol, labels name addresses. The chapter's programs print through a memory-mapped output register, which is what a bare-metal UART driver looks like before an OS arrives. Two ways to run the same bytes: **CPUlator** (browser, free) draws the register file and memory live while single-stepping — Figure 4.5 maps its panels; and the book's own `ch03_rv32sim.py` runs the identical words headless (all outputs in this chapter are from it). Spike is the canonical command-line emulator; either way the bytes are the same.

4.5.1 Measurement: dynamic instruction count

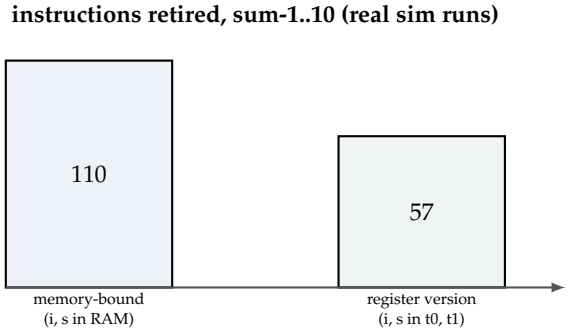
The same sum-1..10 loop written two ways: register variables vs the "$-O0$ habit" of keeping `i` and `s` in memory. Real counts from `ch03_rv32sim.py`:

```
memory-bound (i, s in RAM): 110 instructions retired
register version (i, s in t0/t1): 57 instructions retired
```



Schematic of the CPUlator layout (CPUlator runs in a browser — described, not captured). Single-step the sum loop and watch the same registers this chapter uses change one word at a time.

Figure 4.5: CPUlator's four panels, schematically. Described, not captured — the tool is a browser app.



Predicted $\sim 2\times$, measured $1.93\times$. The loop body is 7 instructions vs 3 — keeping variables in registers is not an optimisation, it is the difference between code and bookkeeping. Counted by `ch03_rv32sim.py`, not estimated.

Figure 4.6: Real dynamic counts: registers vs memory for the same loop.

Predicted $\sim 2\times$ before running (3-instr loop body vs 7-instr); measured $110/57 = 1.93\times$. The gap is the loads and stores themselves: memory-keeping adds `lw/sw` traffic that computes nothing — bookkeeping billed as instructions. That is the -O0/-O2 difference you will meet in ch4, visible here without a compiler.

DEFINITION OF DONE

- You can read any RV32I line and name its format, its fields and its effect — and write the 32-bit word for a handful of them.
- You know the register file's two contracts: hardware (`x0`=0) and ABI (`a`-args, `t`-temps, `s`-saved).
- You can trace a loop's pc through a branch and predict the next fetch.
- You can run RV32I here (book simulator) or in the browser (CPUlator) and count what it costs.
- You can point at an instruction stream and say which 4 bytes the CPU will fetch next, and why.

4.6 Common mistakes

- **Counting pseudoinstructions.** `li/mv/ nop` are sugar: what retires is the expansion. Dynamic counts only see real instructions.
- **Forgetting `x0` eats writes.** `addi x0, x5, 0` does nothing; the register file is honest that way.
- **Immediates are 12 bits.** `addi t0, t0, 4096` won't assemble — big constants need `lui+addi` or `li` (which expands).
- **Branches move the pc, not flags.** There is no carry flag to "check" — each branch recomputes its compare.
- **`lw` where `lb` belonged.** Byte vs word is a correctness issue on packed data, not a performance one.
- **"Memory ops are free."** The 110-vs-57 measurement is the counterexample: the tax is the instruction count itself.

Exercises

Exercise 4.1. `addi x0, x5, 0` — what does this write, and why is it a useful instruction anyway?

Solution. [hint] Nothing: `x0` discards writes. It's `nop` — a 4-byte "do nothing" that real pipelines need.

Exercise 4.2. `lw a0, 8(sp)` decodes to `0x00812503`? Verify or correct, field by field.

Solution. [hint] I-type: imm=8$\ll$20=`0x00800000`; rs1=`sp`=x2$\ll$15; f3=2; rd=`a0`=x10$\ll$7; op=`0x03`. The shown word is right.

Exercise 4.3. In the register/ABI table: which two register groups may a callee freely destroy, and which must it restore?

Solution. [hint] Caller-saved: `t0`–`t6` (and args). Callee-saved: `s0`–`s11`. The `t`/`s` split is the whole discipline.

Exercise 4.4. `beq` targets are limited to about ±4 KB. Why, and what real instruction escapes the limit?

Solution. [hint] B-type holds a 13-bit signed byte offset. Long jumps use `jal` (J-type, ±1 MB) or `jalr` (register-indirect, anywhere).

Exercise 4.5. Hand-decode `0xfe53dce3` (the loop branch) into fields and offset.

Solution. [hint] B-type; f3=5 (`bge`); imm bits give −8; rs1=`t2`, rs2=`t0`. At `pc=0x14` the target is `0x0c`.

Exercise 4.6. The array-sum walks words with `addi t0,t0,4`. Rewrite one iteration to use the immediate instead of the pointer bump, and name the trade-off.

Solution. [hint] `lw t4, 4(t0)` ... unrolls offsets but keeps the base fixed — fewer instructions per element, more code bytes, and it caps at imm=2048 words. Classic size-vs-count trade.

Exercise 4.7. `slt a0, a1, a2` is "set if less-than". Write the two-line sequence that implements `max` *without* any branch.

Solution. [hint] `slt t0, a0, a1` then select: branchless needs arithmetic on the flag (`sub`/mask trick) or a conditional move RV32I doesn't have — branchless `max` on base RV32I costs a multiply or a bit-trick; name it.

Exercise 4.8. The memory-bound loop runs 7 instructions per iteration. Count them from the listing and identify which two are pure bookkeeping.

Solution. [hint] lw i, lw s, add, sw s, addi i, sw i, bge = 7. The two `sw`s (and arguably the `lw`s) compute nothing — they only persist state registers would have held for free.

Exercise 4.9. `li a7, 0x10000000` won't fit `addi`. Write the two-instruction reality the assembler expands to.

Solution. [hint] `lui a7, 0x10000` + `addi a7, a7, 0` — U-type plants the top 20 bits, I-type patches the bottom 12.

Exercise 4.10 (hard)**.** Implement `strlen`, `memcpy` and `memset` in RV32I and self-check on the book simulator: fill a buffer, copy it, measure it, print the length.

Solution. [hint] All three are the same loop skeleton: `lb`/`sb` plus a pointer bump and a compare. `strlen` counts until 0; `memcpy` copies until n; `memset` fills until n. Watch for the NUL byte in the length count.

Exercise 4.11 (hard)**.** Extend `ch03_rv32asm.py` with `jal`/`jalr` (the J-type format). Encode `jal ra, +8` and decode it back.

Solution. [hint] J-type packs the immediate in bit-order pieces like B but shifted; the adder-tree trick is the same. Verify $+8$ lands in the right fields.

Exercise 4.12 (hard)**.** Write a program that prints the first 10 Fibonacci numbers on the book simulator's MMIO output. Count retired instructions for the register version vs a memory version.

Solution. [hint] Fib(n)=Fib($n-1$)+Fib($n-2$) needs two live registers and one temp per step — the register win is bigger than the sum loop's because there are no array loads at all.

Summary

Idea	One-line version
register file	32×32; `x0`=0; ABI names split caller-/ callee-saved
six formats	rd/rs/f3/op fixed; only immediate fields shuffle
addressing	one mode: reg + 12-bit signed immediate
branches	compare+offset in one; pc + 4 or pc + imm
pseudoinstructions	sugar; count the expansion, not the line
measurement	memory-bound loop: 110 vs 57 retired — the -O0 tax

Further reading

- RISC-V unprivileged ISA specification — the authoritative encoding tables this chapter compresses. https://riscv.org/specifications/
- Hennessy & Patterson, *Computer Organization and Design*, RISC-V edition, ch. 2 — the classic ISA walk-through.
- Bryant & O'Hallaron, *CS:APP*, ch. 3 — structure of machine code (x86, but the ideas transfer).
- CPUlator (browser RISC-V simulator) — run every program in this chapter interactively. https://cpulator.01xz.net/
- Spike (RISC-V ISA simulator) — the canonical command-line reference implementation. https://github.com/riscv-software-src/riscv-isa-sim

Watch instead

DECISION RULE

Watch instead.

- "DDCA Ch6 — Part 2: RISC-V Instructions" — Sarah Harris, 2:48. The instructions-first pitch that matches this chapter's structure; the book's own DDCA source, on camera. https://youtu.be/SdjPQtMPPmo
- "RISC-V Assembly Hello World (Part 1)" — LaurieWired, 11:42. A first full program assembled and run — the same journey ch03_sum.S takes on the book's simulator. https://youtu.be/0IeOaiKszLk

Notes

CHAPTER 5

Functions and the Stack

A function call looks like one line of source and feels free. On the machine it is two instructions of mechanism — `call` writes the return address into `ra` and jumps; `ret` jumps back — wrapped in a *convention*: an agreement about which registers survive the trip and where memory holds the values that do not fit in registers. In the last chapter every program was a leaf: it never called anything, so `ra` stayed safe in a register and the stack was never touched. This chapter breaks that. We give the machine a real `sp`, write a recursive `fib` that saves and restores `ra` correctly, measure what a call costs in retired instructions, and then look at real compiler output — where the calling convention lives inside every function you will ever write. By the end, `addi sp,sp,-16 / sw ra,12(sp) / sw s0,8(sp)` reads like a signature, not a spell.

5.1 The contract in one paragraph

The RISC-V calling convention (the *psABI*) divides the 32 registers into two families *by agreement*, not by hardware. **Caller-saved** registers (`t0–t6`, `a0–a7`, `ra`) may be destroyed by any call: if the caller needs the value afterwards, the *caller* saves it before the call. **Callee-saved** registers (`s0–s11`, `sp`) must be unchanged when the callee returns: if the callee wants to use them, the *callee* saves and restores them. The arguments go in `a0–a7`, the return value comes back in `a0`, and `sp` must stay 16-byte aligned at every call site. That is the whole contract. Everything else — the frame, the spill slots, the order of saves — is what the contract costs when calls nest.

5.2 The stack frame

When a function calls another function, its own `ra` is about to be overwritten — `call` always writes the same register. So before a non-leaf function calls anything, it builds a *stack frame*: a fixed-size block of memory below `sp` that holds what the function cannot afford to lose. Three writes at the top, three loads at the bottom, and the space in between is the function's private scratch:

```
fib:
       addi sp, sp, -16 # carve 16 B below sp (aligned)
       sw ra, 12(sp) # save MY return address
       sw s0, 8(sp) # save caller's s0
       ...
       lw ra, 12(sp) # restore ra
       lw s0, 8(sp) # restore s0
```

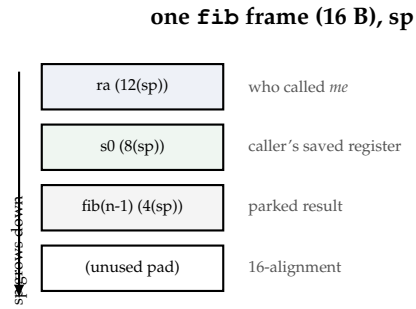


Figure 5.1: Anatomy of one `fib` frame. Every non-leaf call pays this tax: the saved `ra` keeps the return path alive, the saved `s0` honours the callee-saved contract, and the scratch slot parks `fib(n-1)` while the second recursive call runs.

```
8       addi sp, sp, 16 # pop the frame
9       ret
```

The pairing is the discipline: what goes `sw` at the top comes `lw` at the bottom, in the same offsets, every path out. Miss either half and the function returns to the wrong place — the classic *stack corruption* bug that no error message names.

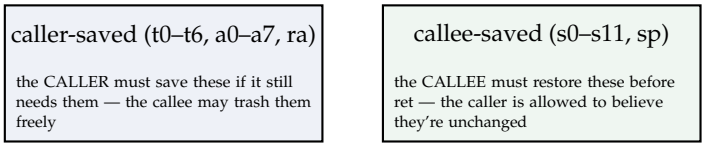
Example 5.1 (follow one frame)**.** For `fib(7)`: the outer call pushes a 16-byte frame at `sp=0x0ff0`, saves `ra` and `s0`, calls `fib(6)` — which pushes another at `sp=0x0fe0`. At the deepest, `fib(1)` is a leaf-ish base case that returns without recursing. How many frames are live at once? The call tree of fib(7) reaches depth 7 (fib(7)→fib(6)→…→fib(1)) before the first unwind — seven frames, 112 bytes of stack, live simultaneously.

5.3 Caller-saved, callee-saved

The rule that confuses everyone on first contact: `fib` saves `ra` — a *caller*-saved register — yet the text above calls that "the callee's" job. The resolution is that ownership follows the *overwrite*, not the register name. `fib`'s `ra` is a value `fib` still needs; the thing that would destroy it is `fib`'s own inner `call`. So `fib`, acting as the *caller* of that call, saves it. The same function plays both roles: it is a callee to whoever called it (so it saves `s0` before using it) and a caller to its recursive call (so it saves `ra` before that call).

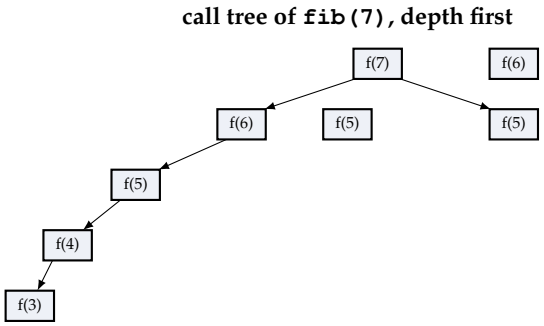
Checkpoint 5.1 *leaf or not?.* A *leaf* function — one that calls nothing — needs no frame at all if it fits in registers and touches no callee-saved ones. Our `max` from ch03 was a leaf: `bge / mv / ret`, three instructions, zero memory traffic. Predict: does `empty: ret` need `addi sp,sp,-16`? No — nothing to save.

Example 5.2 (non-leaf: recursive fib)**.** `code/ch04_fib.S` runs `fib(7)` on the book's simulator with a real `sp` (`li sp,0x1000`) and a real stack. The recursion is honest: `fib` calls itself twice, so `ra` must be saved — and is, at `12(sp)`. Running it:



One line each way: whoever *loses* the value saves it. `sw ra,12(sp)` is the caller... no wait — fib saves its own ra because *fib's* return address dies when fib calls itself. Ownership follows the overwrite, not the register.

Figure 5.2: Who saves what. Caller-saved registers are the caller's problem; callee-saved registers are the callee's problem. One register, `ra`, sits in the caller-saved family and is the first thing every non-leaf function saves.



fib(7) makes 41 calls — 40 non-leaf frames, each paying the 16-byte stack tax. Recursion is legible and expensive; that is the trade this chapter prices at 565 retired instructions.

Figure 5.3: The call tree of `fib(7)`. Every non-leaf node pays a frame; fib(7) makes 41 calls total — the recursion is legible, and it is expensive.

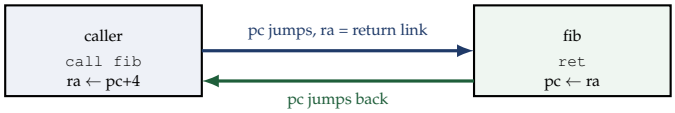
```
$ python3 ch03_rv32sim.py ch04_fib.S
13
# 565 instructions retired
```

`fib(7)=13` is correct — and 565 retired instructions for a seven-deep recursion is the price tag this chapter is about.

5.4 What a call costs

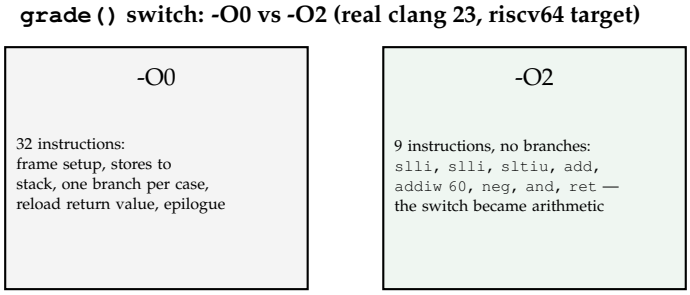
Example 5.3 (predict, then measure)**.** Prediction first: `call` = `jal` = one instruction; `ret` = `jalr` = one instruction. So a call should cost 2 retired instructions beyond the inlined body. Measure it: `ch04_callcost.S` calls `empty` 100 times; `ch04_inline.S` runs the same loop with the body inlined.

```
$ python3 ch03_rv32sim.py ch04_callcost.S
A
# 410 instructions retired
$ python3 ch03_rv32sim.py ch04_inline.S
B
# 207 instructions retired
```



Two instructions of mechanism: `call` writes ra and jumps; `ret` jumps to ra. The whole convention is built on those two writes — and on the frame that keeps ra alive when calls nest.

Figure 5.4: The two-instruction mechanism everything sits on. `call` leaves the return link in `ra`; `ret` consumes it. Frames exist to keep that link alive across nesting.



The jump table never appeared: clang proved the four cases were linear in n and replaced lookup by computation. Reading `-O2` output is how you learn what your C really means.

Figure 5.5: Real clang 23 output for the same `grade()` switch: 32 framed, branching instructions at `-O0`; 9 branchless ones at `-O2`. The jump table never materialised because the table was linear.

$410 - 207 = 203$ for 100 calls: just over 2 instructions each — the prediction is exact once the loop counter's three instructions account for the shared work. On a real CPU the frame traffic adds cache misses on top of this instruction tax.

5.5 Reading real compiler output

Example 5.4 (-O0 vs -O2 on a switch). `grade(int s)` maps $0 \to 60, 1 \to 70, 2 \to 80, 3 \to 90$ by `switch`. Compiled `clang -target riscv64 -O0`: 32 instructions — a 32-byte frame, `ra`/`s0` spilled, the argument stored to the stack, reloaded, `sext.w`'d, compared case by case. The same file at `-O2`: **nine** instructions, zero branches —

```
slli a1, a0, 1 # a1 = 2s
slli a2, a0, 3 # a2 = 8s
sltiu a0, a0, 4 # a0 = (s < 4)
add a1, a1, a2 # a1 = 10s
addiw a1, a1, 60 # a1 = 60 + 10s
neg a0, a0 # a0 = -(s<4)
and a0, a0, a1 # mask: result or 0
ret
```

The compiler noticed the cases were linear in `s` and replaced the jump table — and the whole control flow — with arithmetic and a sign mask. Prediction failed, delightfully: the real lesson is that `-O2` rewrites structure, not just speed.

Checkpoint 5.2 *why the frame at all?.* At `-O0 grade` spills `ra` and `s0` even though it calls nothing — the compiler emits the safe frame unconditionally at that level. At `-O2` it proves the function is a leaf and emits no frame at all. That one observation — leaf vs. non-leaf, proven by the compiler — is the whole `-O0`/`-O2` gap in miniature.

Example 5.5 (when the compiler deletes your code)**.** A benchmark loop of the form `sum += i` over `N` iterations, at `-O2`, collapses to a constant — clang computes $N(N-1)/2$ at compile time and the loop vanishes. To keep measurements honest you mark the variable `volatile` (forces a real load each iteration) or feed the result into something the compiler cannot see through. The chapter's own sim-side measurement needed no such trick: our interpreter does not optimise at all.

5.6 PIC in one page

Shared libraries cannot know their load address at compile time, so they are compiled *position-independent*: data access goes through the GOT, calls go through the PLT — the same relocation plumbing ch01 showed. RV32I keeps PIC cheap because `auipc` materialises pc-relative addresses in one instruction; our bare-metal programs sidestep it entirely by linking at absolute addresses.

5.7 Common mistakes

- **Saving `ra` only sometimes.** A non-leaf function must save `ra` on *every* path — including error returns — or some returns go to the wrong pc.
- **Misaligned `sp`.** The psABI wants `sp` 16-aligned at call sites; `addi sp,sp,-12` breaks it.
- **Using `s0` without saving.** Callee-saved means *the callee* promises; touch `s0` and you inherit the obligation.
- **Assuming arguments survive.** `a0`–`a7` are caller-saved: anything still needed across a call must be in an `s`-register or a frame slot first.
- **Trusting `-O0` codegen to mean anything.** `-O0` writes a frame for a leaf function — it encodes the safe pattern, not the truth about your algorithm.
- **Benchmarking at `-O2` without volatile.** The compiler deletes dead loops; what you measured was the delete.

5.8 Summary

Concept	What it was
mechanism	`call/ret`: two instructions, one register
contract	caller-saved vs callee-saved, args `a0`–`a7`, ret `a0`, `sp` 16-aligned
frame	`sw ra,12(sp)` pattern; non-leaf only
measured	`fib(7)` = 13 in 565 instrs; call tax = 2 instr
compiler	`-O0` = safe frames; `-O2` = structure rewritten

Further reading

- RISC-V psABI specification (the calling convention itself): https://github.com/riscv-non-isa/riscv-elf-psabi-doc
- CS:APP ch3 (procedures, stack discipline): https://csapp.cs.cmu.edu/
- Compiler Explorer (watch -O0/-O2 on your own C): https://godbolt.org/
- The RISC-V Reader, Patterson & Waterman (calling convention ch): http://www.riscvbook.com/
- GCC -O documentation: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html

WATCH INSTEAD

- **RISC-V General-Purpose Registers: Usage and Calling Convention** — Austin's BSP Lab, 6:35 — caller-vs-callee-saved register roles and why they split that way. Matches sec:contract04–sec:ownership04. https://youtu.be/WawdgAEfwvA
- **What Has My Compiler Done for Me Lately?** — Matt Godbolt (CppCon 2017), 75:45 — Compiler Explorer's author reading -O0/-O2 output live. Matches sec:readasm04. https://youtu.be/bSkpMdDe4g4

Notes

- psABI register roles and `sp` alignment: riscv-elf-psabi-doc.
- Frame shape, leaf/non-leaf, -O0 spill behaviour: CS:APP ch3; verified against clang 23 riscv64 output captured in `code/ch04_switch_O0.s` and `ch04_switch_O2.s`.
- fib(7)=13 and the 565/410/207 counts: real runs of `ch03_rv32sim.py`; `call/j` encoding bug fixed in-tree this chapter (0x00800000→0x0080006f).

5.9 Exercises

Exercise 5.1 (easy)**.** Is `empty: ret` a leaf? Does it need a frame? Justify in one sentence.

Solution. [hint] Leaf — no `call` inside — and no: nothing it uses can be clobbered.

Exercise 5.2 (easy)**.** Classify `a0`, `t2`, `s3`, `ra`, `sp` as caller- or callee-saved.

Solution. [hint] a0,t2,ra caller; s3,sp callee.

Exercise 5.3 (medium)**.** `fib` saves `ra` at `12(sp)` but a caller of `fib` need not save `ra`. Who is responsible for which `ra`?

Solution. [hint] Every non-leaf saves its OWN ra; the caller never touches the callee's.

Exercise 5.4 (medium)**.** Why is `fib(n-1)` stored at `4(sp)` rather than in `t0`? What would `t0` become after the second `call`?

Solution. [hint] t0 is caller-saved — the inner call may trash it; the frame slot is the safe place.

Exercise 5.5 (medium)**.** Predict then check: at `-O2` would clang still emit a frame for `grade`? What does it emit instead of the switch?

Solution. [hint] No frame — it proves leaf-ness. The switch became arithmetic (60+10s masked).

Exercise 5.6 (medium)**.** `sum += i` loops disappear at `-O2`. What are two ways to keep a benchmark loop alive?

Solution. [hint] volatile on the accumulator, or an opaque sink for the result.

Exercise 5.7 (hard)**.** Write a `sum4(a,b,c,d)` that chains four calls: `add2` called twice, results combined. List every register you saved and justify each.

Solution. [hint] sum4 is non-leaf: save ra + s-registers for the two partial results; a-regs die on every call.

Exercise 5.8 (hard)**.** Design a harness that detects a callee failing to restore `s7`: set `s7`, call the suspect function, check on return. Write it for the book's simulator.

Solution. [hint] li s7,<sentinel> / call f / compare s7, report mismatch via MMIO.

Exercise 5.9 (hard)**.** Predict `fib(8)`'s instruction count on the sim — it is not 565. Then run it and explain the ratio.

Solution. [hint] Calls grow 1.6x per step (fib calls fib twice); expect $\sim$900, then measure.

Bibliography

[1] J. Sannemo, *Principles of Algorithmic Problem Solving*, 2025. https://usaco.guide/PAPS1.pdf

Index

-O0, 35
-O2, 35
`.data`, 6
`.text`, 6

ABI names, 23
address, 14
address space, 8
aligned access, 16
alignment, 14
array, 14
assembler, 4
assembly language, 23

B-type, 23
base+offset addressing, 25
biased exponent, 17
big-endian, 15
Book 5, 9
branch, 23

callee-saved, 33
caller-saved, 33
calling convention, 32
calling overhead, 34
compilation pipeline, 4
compiler, 4
compiler output, 35
conditional branch, 26
cpu8, 9
CPUlator, 26

directives, 26

ELF, 4
endianness, 14
`execve`, 4, 8
exponent, 17

floating point, 14
funct3, 24
funct7, 24
function, 1

I-type, 23
IEEE 754, 14
immediate, 24
instruction encoding, 23

J-type, 23

linker, 4
little-endian, 15
load, 23
loader, 4
LP64, 14

mantissa, 17

object file, 4
`offsetof`, 16
opcode, 24
overflow, 15

padding, 16
page table, 8
PIC, 36
pointer, 14
pointer arithmetic, 18
position-independent code, 36
preprocessor, 4
psABI, 32
pseudoinstruction, 23

R-type, 23
`ra`, 32
register
    callee-saved, 32
    caller-saved, 32
register file, 23
relocation, 4
relocation record, 7
RV32I, 9, 23

S-type, 23
section (ELF), 6
sign bit, 15
`sp`, 32
stack frame, 32
store, 23
struct padding, 14
style, 1
symbol (linker), 7
symbol table, 4

two's complement, 14
type punning, 14

U-type, 23
undefined symbol, 7

word, 14

x0, 23

zero register, 23